Streamlit Tutorial Series

A Complete Guide for Students

From Hello World to Full Image Classification App

Examples 01–12 | Keras Integration | GitHub & Streamlit Cloud Deployment

1. What is Streamlit?

ما هي Streamlit؟

Streamlit مكتبة Python مجانية ومفتوحة المصدر بتحوّل ملف Python عادي بامتداد .py إلى تطبيق ويب تفاعلي، من غير الحاجة إلى كتابة HTML أو CSS أو JavaScript. وهي موجهة خصوصًا لعلماء البيانات ومهندسي تعلم الآلة لمشاركة أعمالهم في صورة أدوات تفاعلية.

باستخدام Streamlit يمكن إنشاء لوحات معلومات وتطبيقات بيانات باستخدام Python فقط، وعرض الرسوم والجداول والصور وتوقعات النماذج، واستقبال مدخلات المستخدم مثل الملفات والنصوص والـ sliders والقوائم المنسدلة، ثم نشر التطبيق عبر Streamlit Cloud.

Key Facts About Streamlit

- Created by: Streamlit Inc. (now part of Snowflake)
- Language: Python only — no web development knowledge required
- Execution model: the entire script reruns top-to-bottom on every user interaction
- Deployment: one command to run locally; one click to deploy to the cloud
- License: Apache 2.0 — completely free and open source

2. Installation & Setup

التثبيت والإعداد

المتطلبات المذكورة في المحاضرة: Python 3.8 أو أحدث، وأداة pip التي تأتي مع Python، ومحرر أكواد مثل VS Code.

2.1 Install Streamlit

```
# Open your terminal and run:
pip install streamlit

# Verify the installation:
streamlit --version
```

2.2 Install All Required Packages

لتشغيل الأمثلة الكاملة، بما فيها تصنيف الصور، ثبّت الحزم التالية:

```
pip install streamlit pandas numpy matplotlib seaborn
pip install Pillow scikit-learn tensorflow opencv-python-headless
```

2.3 Your First Run

أنشئ ملفًا باسم app.py داخل مجلد المشروع، ثم شغّله من Terminal. يفتح Streamlit المتصفح تلقائيًا على العنوان المحلي الموضح.

```
# Create a file called app.py, then run:
streamlit run app.py

# Streamlit opens your browser automatically at:
# http://localhost:8501
```

3. How Streamlit Works

آلية عمل Streamlit

فهم طريقة تنفيذ Streamlit مهم جدًا. عند أي تفاعل من المستخدم، مثل الضغط على زر أو تحريك Slider أو كتابة نص، يعيد Streamlit تشغيل ملف Python بالكامل من أول سطر إلى آخر سطر.

3.1 The Rerun Model

• تكتب التطبيق كسكريبت Python عادي من أعلى إلى أسفل.

• كل Widget مثل st.button وst.slider ترجع القيمة الحالية.

• تستخدم statements if للتعامل مع قيم عناصر التفاعل.

• لا تحتاج إلى callbacks أو handlers event في النمط الموضح بالمحاضرة.

```
import streamlit as st

# This runs every time anything changes
name = st.text_input("Enter your name")

# React immediately to the input
if name:
    st.write(f"Hello, {name}!")
```

3.2 Session State

بسبب إعادة تشغيل السكريبت، يتم إعادة ضبط متغيرات Python العادية. للاحتفاظ بقيمة بين مرات التشغيل نستخدم st.session_state. المثال التالي ينشئ عدادًا مرة واحدة، ثم يزيده عند الضغط على الزر ويعرض قيمته.

```
# Initialize once
if "counter" not in st.session_state:
    st.session_state.counter = 0

# Update on button click
if st.button("Click me"):
    st.session_state.counter += 1

# Read the persisted value
st.write(f"Count: {st.session_state.counter}")
```

3.3 Multi-Page Apps

يتعرف Streamlit تلقائيًا على مجلد اسمه pages/ بجوار app.py، ويضيف روابط تنقل في الشريط الجانبي لكل ملف Python موجود داخله.

```
my_app/
├── app.py          <- run this file
└── pages/
    ├── 1_Settings.py
    ├── 2_Data.py
    └── 3_Results.py
```

4. Example 01 – Hello Streamlit

أول تطبيق Streamlit

أهداف التعلّم:

• Understand what Streamlit is and how to run it

• Display a title, header, subheader, and text on the page

• Run a Streamlit app from the terminal

شرح الفكرة وطريقة العمل:

الأهداف: فهم Streamlit وتشغيله، وعرض title وheader وsubheader و.text نبدأ بأبسط تطبيق لفهم الهيكل: استيراد المكتبة، استخدام الدوال التي تبدأ بـ .st، ثم تشغيل الملف. st.title() للعنوان الأكبر، وst.header() وst.subheader() للعناوين الأصغر، وst.write() لعرض النصوص والبيانات.

Complete Code

```
import streamlit as st

# Biggest heading on the page
st.title("Hello, Streamlit!")

# Section heading
st.header("Welcome to the tutorial")

# Smaller heading
st.subheader("We will build apps step by step.")

# General purpose text display
st.write("This is my first Streamlit app.")
```

Key Functions

• st.title("text") — Biggest heading on the page

• st.header("text") — Section heading

• st.subheader("text") — Smaller subheading

• st.write("text") — General purpose; displays text, numbers, DataFrames

Student Exercise

• Change the title to your own name

• Add a subheader with today's date

• Add st.write() with a short introduction about yourself

5. Example 02 – Text & Markdown

تنسيق النصوص وMarkdown

أهداف التعلّم:

• Format text using Markdown (bold, italic, lists, headers)

• Add colored text using HTML inside Markdown

• Display code blocks with syntax highlighting

• Use special message boxes: info, success, warning, error

شرح الفكرة وطريقة العمل:

الأهداف: تنسيق النص باستخدام Markdown مثل bold وitalic والقوائم والعناوين، وإضافة ألوان باستخدام HTML داخل Markdown، وعرض الأكواد مع highlighting syntax، واستخدام صناديق الرسائل info وsuccess وwarning وerror. الدالة st.markdown() تقبل صيغة Markdown. استخدام True=unsafe_allow_html يسمح بتمرير HTML داخل Markdown. الدالة st.code() تعرض الكود بتنسيق واضح.

Complete Code

```
import streamlit as st

st.title("Example 02 - Text & Markdown")

# Markdown formatting
st.markdown("**This text is bold**")
st.markdown("*This text is italic*")
st.markdown("## Section Header")

# Bullet list
st.markdown("""
### My List
- Item one
- Item two
- Item three
""")

# Colored text using HTML
st.markdown("<p style='color: red;'>Red text</p>",
        unsafe_allow_html=True)

# Code block
st.code("import streamlit as st", language="python")

# Message boxes
st.info("This is an info box")
st.success("This is a success box")
st.warning("This is a warning box")
st.error("This is an error box")
```

Key Functions

• st.markdown("text") — Formatted Markdown text

• st.code("code", language="python") — Syntax-highlighted code block

• st.info("text") — Blue info box

• st.success("text") — Green success box

• st.warning("text") — Yellow warning box

• st.error("text") — Red error box

Student Exercise

• Add your name as a large colored heading

• Write a numbered list of 3 things you learned today

• Display a Python code snippet of your choice

• Add a green success box that says "I completed Example 02!"

6. Example 03 – Widgets Basics

عناصر التفاعل الأساسية

أهداف التعلّم:

• Add interactive buttons, sliders, and text inputs

• Use checkboxes, radio buttons, and dropdown menus

• Read widget values and react to them with if statements

• Build a mini profile card app using all widgets

شرح الفكرة وطريقة العمل:

الأهداف: إضافة الأزرار والـ sliders وحقول النص، واستخدام checkboxes وbuttons radio وmenus dropdown، وقراءة القيم والتعامل معها باستخدام if، وإنشاء بطاقة Profile بسيطة. كل Widget عبارة عن استدعاء دالة يعيد القيمة الحالية. النمط العام: st.widget_name("label", ...options). عند تغيير القيمة يعاد تشغيل السكريبت وتصبح القيمة الجديدة متاحة تلقائيًا.

Complete Code

```
import streamlit as st

# Button — returns True when clicked
if st.button("Click Me!"):
    st.success("You clicked the button!")

# Slider — returns the selected number
age = st.slider("Select your age", 0, 100, 25)
st.write(f"You selected: {age} years old")

# Text input
name = st.text_input("Enter your name")
if name:
    st.success(f"Hello, {name}!")

# Checkbox
agree = st.checkbox("I agree to the terms")

# Radio buttons
subject = st.radio("Favourite subject",
    ["Math", "Science", "CS"])

# Selectbox (dropdown)
city = st.selectbox("Select your city",
    ["Cairo", "Dubai", "London"])

st.write(f"City: {city}, Subject: {subject}")
```

Key Functions

• st.button("label") — Returns True when clicked

• st.slider("label", min, max, default) — Number picker via drag

• st.text_input("label") — Single-line text box

• st.text_area("label") — Multi-line text box

• st.checkbox("label") — Returns True/False

• st.radio("label", [options]) — Pick one from a list

• st.selectbox("label", [options]) — Dropdown menu

• st.number_input("label", min, max) — Precise number entry

Student Exercise

• Build a BMI Calculator: sliders for height and weight, button to calculate

• Formula: BMI = weight / (height/100)^2

• Show the result and a colored message box based on BMI category

7. Example 04 – Layout: Columns, Sidebar, Tabs

تنظيم الواجهة

أهداف التعلّم:

• Arrange content side by side using columns

• Move controls to a sidebar panel

• Hide/show content with expanders

• Organize content into clickable tabs

• Display key numbers with metric cards

شرح الفكرة وطريقة العمل:

الأهداف: ترتيب المحتوى جنبًا إلى جنب باستخدام columns، ونقل عناصر التحكم إلى sidebar، وإظهار وإخفاء المحتوى باستخدام expanders، وتنظيم المحتوى في tabs، وعرض الأرقام المهمة باستخدام metric cards. بدون أدوات Layout يظهر كل شيء في عمود واحد. st.columns(2) تنشئ عمودين متساويين، ويمكن استخدام col1: with لوضع المحتوى داخل عمود. المحتوى داخل st.sidebar: with يظهر في الشريط الجانبي. st.tabs() تنشئ تبويبات، وst.expander() ينشئ قسمًا قابلًا للطي.

Complete Code

```
import streamlit as st

# Side-by-side columns
col1, col2, col3 = st.columns(3)
with col1:
   st.metric("Students", 120)
with col2:
   st.metric("Average", "87.5", delta="+3.2")
with col3:
   st.metric("Pass Rate", "94%")

# Sidebar
with st.sidebar:
   st.header("Settings")
   subject = st.selectbox("Subject", ["Math","Science"])
   students = st.slider("Students", 10, 100, 30)

# Tabs
tab1, tab2 = st.tabs(["Results", "Details"])
with tab1:
   st.write("Results tab content")
with tab2:
   st.write("Details tab content")

# Expander
with st.expander("Click to see more"):
   st.write("Hidden content revealed!")
```

Key Functions

• st.columns(n) — Creates n side-by-side columns

• st.columns([1,2,1]) — Columns with custom widths
• st.sidebar — Left-side panel for controls
• st.tabs(["A","B"]) — Clickable tab sections
• st.expander("label") — Collapsible section
• st.container() — Invisible grouping box
• st.metric(label, value, delta) — Dashboard number card

Student Exercise

• Build a Student Report Card app
• Put student name, grade, and subject in the sidebar
• Show 4 metric cards: Score, Rank, Attendance, Grade
• Use 3 tabs: Results / Comments / Settings

8. Example 05 – Displaying Data

عرض البيانات

أهداف التعلّم:

• Display pandas DataFrames as interactive sortable tables
• Use metric cards for key numbers
• Filter data interactively with sidebar widgets
• Display JSON and dictionary data

شرح الفكرة وطريقة العمل:

الأهداف: عرض pandas DataFrames كجداول تفاعلية قابلة للترتيب، واستخدام metric cards للأرقام المهمة، وتصفية البيانات باستخدام عناصر sidebar، وعرض JSON و dictionary. st.dataframe() يعرض جدولًا تفاعليًا يمكن ترتيبه والتمرير داخله، بينما st.table() يعرض جدولًا ثابتًا. st.metric() يعرض رقمًا واحدًا مع عنوان. يمكن دمج Widgets مع DataFrames لعمل تصفية تفاعلية.

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np

# Create sample data
df = pd.DataFrame({
   "Name"   : ["Alice", "Bob", "Carol"],
   "Math"   : [90, 75, 88],
   "Science": [85, 80, 92],
   "Grade"  : ["A", "B", "A"],
})

# Interactive sortable table
st.dataframe(df, use_container_width=True)

# Metric cards
col1, col2, col3 = st.columns(3)
with col1:
   st.metric("Total Students", len(df))
with col2:
   st.metric("Average Math", df["Math"].mean())
with col3:
   st.metric("Top Score", df["Math"].max())

# Filter with multiselect
grades = st.multiselect("Filter by Grade",
   df["Grade"].unique(), default=["A","B"])
filtered = df[df["Grade"].isin(grades)]
st.dataframe(filtered)
```

Key Functions

• st.dataframe(df) — Interactive sortable table

• st.table(df) — Static clean table

• st.metric(label, value, delta) — Single number card with change

• st.json(dict) — Pretty-printed dictionary / JSON

• st.multiselect("label", options) — Dropdown with multiple selections

Student Exercise

• Create a DataFrame of 15 students with 4 subjects

• Show 4 metric cards: total students, highest score, lowest score, class average

• Add a sidebar slider to filter by minimum score

• Add a multiselect to filter by grade (A, B, C)

9. Example 06 – Charts & Plots

الرسوم البيانية

أهداف التعلّم:

• Use Streamlit built-in charts (line, bar, area)

• Display Matplotlib and Seaborn figures

• Connect charts to widgets so they update live

• Plot training accuracy and loss curves

شرح الفكرة وطريقة العمل:

الأهداف: استخدام الرسوم المدمجة في Streamlit مثل line وbar وarea، وعرض رسومات Matplotlib وSeaborn، وربط الرسوم بالـ widgets لتتحدث مباشرة، ورسم منحنيات دقة التدريب والخسارة. الرسوم تجعل الأنماط أوضح من الأرقام في الجداول. يمكن استخدام st.line_chart وst.bar_chart مباشرة، أو إنشاء figure Matplotlib ثم عرضها باستخدام st.pyplot(fig). ونفس النمط مع Seaborn.

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Built-in line chart (one line of code)
history = pd.DataFrame({
   "Train Accuracy": [0.5, 0.7, 0.85, 0.91, 0.94],
   "Val Accuracy"  : [0.48, 0.65, 0.80, 0.87, 0.90],
})
st.line_chart(history)

# Matplotlib figure (full control)
fig, ax = plt.subplots(figsize=(8, 4))
models = ["SVM", "KNN", "RF", "ANN", "CNN", "MobileNetV2"]
accs   = [72, 68, 75, 83, 88, 91]
ax.bar(models, accs, color="steelblue")
ax.set_title("Model Accuracy Comparison")
ax.set_ylabel("Accuracy (%)")
ax.set_ylim(0, 110)
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
st.pyplot(fig)
```

Key Functions

• st.line_chart(df) — Quick line chart from DataFrame

• st.bar_chart(df) — Quick bar chart

• st.area_chart(df) — Quick area/filled chart

• st.pyplot(fig) — Display any Matplotlib/Seaborn figure

Student Exercise

• Create fake training history for 3 models: ANN, CNN, MobileNetV2

• Show a line chart of all 3 validation accuracies on one chart

• Add a selectbox to switch between Accuracy and Loss views

• Add a slider to control how many epochs to display

10. Example 07 – File Upload

رفع الملفات

أهداف التعلّم:

• Let users upload a CSV file and display it as a table

• Let users upload an image and show it

• Read uploaded files into pandas or NumPy

• Show file information (name, size, type)

• Accept multiple files at once

شرح الفكرة وطريقة العمل:

الأهداف: رفع CSV وعرضه كجدول، ورفع صورة وعرضها، وقراءة الملفات باستخدام pandas أو NumPy، وعرض معلومات الملف مثل الاسم والحجم والنوع، وقبول عدة ملفات. st.file_uploader() يعيد كائن ملف عند الرفع أو None إذا لم يتم رفع ملف. يجب التأكد من أن uploaded_file ليس None قبل القراءة. الصور تُفتح باستخدام PIL.Image.open()، وملفات CSV باستخدام pd.read_csv().

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np
from PIL import Image

# Upload a CSV file
csv_file = st.file_uploader("Upload CSV", type=["csv"])
if csv_file is not None:
    df = pd.read_csv(csv_file)
    st.success(f"Loaded {len(df)} rows")
    st.dataframe(df.head())

# Upload an image
img_file = st.file_uploader("Upload image",
    type=["jpg", "jpeg", "png"])
if img_file is not None:
    image = Image.open(img_file)
    img_array = np.array(image)
    st.image(image, caption="Uploaded Image", width=300)
    st.write(f"Size: {image.width} x {image.height} px")
    st.write(f"Shape: {img_array.shape}")

# Multiple files
files = st.file_uploader("Multiple images",
    type=["jpg","png"], accept_multiple_files=True)
if files:
    cols = st.columns(3)
    for i, f in enumerate(files):
        cols[i%3].image(Image.open(f), use_column_width=True)
```

Key Functions

• st.file_uploader("label", type=[...]) — File upload widget

• accept_multiple_files=True — Allow selecting multiple files

• Image.open(file) — Open uploaded image with PIL

• pd.read_csv(file) — Read uploaded CSV into DataFrame

• np.array(image) — Convert PIL image to NumPy array

• st.image(image, width=300) — Display an image on the page

Student Exercise

• Build an Image Info App: upload an image and show width, height, channels, file size

• Add a slider to resize the display width (100 to 800 px)
• Add a checkbox "Convert to Grayscale" and show the grayscale version if checked

11. Example 08 – Session State

الاحتفاظ بالحالة بين مرات التشغيل

أهداف التعلّم:

• Understand why Streamlit reruns the script on every interaction
• Use st.session_state to remember values between reruns
• Build a counter that actually works
• Build a to-do list app
• Build a multi-step wizard form

شرح الفكرة وطريقة العمل:

الأهداف: فهم سبب إعادة تشغيل Streamlit للسكريبت، واستخدام st.session_state للاحتفاظ بالقيم، وإنشاء counter يعمل، وتطبيق list To-do، ونموذج متعدد الخطوات. المشكلة أن كل تفاعل يعيد تشغيل السكريبت فتُعاد تهيئة المتغيرات العادية. State Session عبارة عن dictionary تستمر قيمه بين مرات إعادة التشغيل. نهيئ المفتاح إذا لم يكن موجودًا، ثم نقرأه أو نحدّثه. ويمكن فرض إعادة تشغيل فورية باستخدام st.rerun().

Complete Code

```
import streamlit as st

# Step 1: Initialize the counter once
if "counter" not in st.session_state:
    st.session_state.counter = 0

# Step 2: Buttons to change the value
col1, col2, col3 = st.columns(3)
with col1:
    if st.button("+ Increment"):
        st.session_state.counter += 1
with col2:
    if st.button("- Decrement"):
        st.session_state.counter -= 1
with col3:
    if st.button("Reset"):
        st.session_state.counter = 0

# Step 3: Display the persisted value
st.metric("Current Count", st.session_state.counter)

# To-do list example
if "tasks" not in st.session_state:
    st.session_state.tasks = []

new_task = st.text_input("New task")
if st.button("Add Task") and new_task:
    st.session_state.tasks.append(new_task)

for task in st.session_state.tasks:
    st.write(f"* {task}")
```

Key Functions

• st.session_state.key = value — Store a value that survives reruns

• st.session_state.key — Read the stored value

• "key" not in st.session_state — Check if key exists before initializing

• st.rerun() — Force the script to rerun immediately

• key="unique_id" — Give each widget a unique key when needed

Student Exercise

• Build a Quiz App with 3 questions

• Show one question at a time using session state

• Track the score in session state

• Show the final score and a Restart button at the end

12. Example 09 – Forms

Forms النماذج

أهداف التعلّم:

• Group multiple widgets inside a form

• Submit all inputs at once with one button

• Prevent the app from rerunning on every small change

• Build a login form, search form, and model config form

شرح الفكرة وطريقة العمل:

الأهداف: تجميع عدة Widgets داخل Form، وإرسال كل المدخلات بزر واحد، ومنع إعادة التشغيل مع كل تغيير صغير، وبناء نماذج تسجيل ودخول وبحث وإعداد نموذج. من غير Form يعاد تشغيل التطبيق عند لمس أي Widget. أما Form فيجمع المدخلات ولا يرسلها إلا عند الضغط على Submit. نستخدم st.form with("unique_name")، ويكون آخر عنصر داخل النموذج. تتم معالجة النتيجة خارج كتلة النموذج باستخدام st.form_submit_button("label") if submitted:.

Complete Code

```
import streamlit as st

# Basic registration form
with st.form("registration_form"):
    st.subheader("Student Registration")
    name    = st.text_input("Full Name")
    age     = st.number_input("Age", 15, 60, 20)
    major   = st.selectbox("Major",
        ["AI", "CS", "Engineering"])
    agree   = st.checkbox("I agree to the terms")
    submitted = st.form_submit_button("Register")

# Handle the result OUTSIDE the form
if submitted:
    if not name.strip():
        st.error("Please enter your name")
    elif not agree:
        st.error("Please agree to the terms")
    else:
        st.success(f"Welcome, {name}!")

# Login form with password
with st.form("login_form"):
    username = st.text_input("Username")
    password = st.text_input("Password",
        type="password")  # hides the text
    login = st.form_submit_button("Login")

if login:
    if username == "admin" and password == "1234":
        st.success("Login successful!")
    else:
        st.error("Wrong credentials")
```

Key Functions

• with st.form("name"): — Create a form; nothing inside reruns until submit

• st.form_submit_button("label") — The submit button; must be inside the form

• type="password" — Hides text input for passwords

• if submitted: — Handle result OUTSIDE the form block

Student Exercise

• Build a BMI Calculator Form with name, height, weight, gender

• Only calculate when Submit is clicked

• Show BMI value and a colored message box based on category

• Categories: Underweight < 18.5 | Normal 18.5-25 | Overweight 25-30 | Obese > 30

13. Example 10 – Multi-Page App

تطبيق متعدد الصفحات

أهداف التعلّم:

• Split a Streamlit app into multiple pages

• Share data between pages using session state

• Understand how automatic sidebar navigation works

• Build a 3-page Student Portal app

شرح الفكرة وطريقة العمل:

الأهداف: تقسيم التطبيق إلى صفحات، ومشاركة البيانات بينها باستخدام state session، وفهم التنقل التلقائي في sidebar، وبناء Portal Student من 3 صفحات. التطبيقات الكبيرة لا يناسبها ملف واحد. ينشئ Streamlit التنقل تلقائيًا لكل ملف .py داخل مجلد pages/. يمكن التحكم في الترتيب بترقيم الملفات. يجب تشغيل التطبيق من app.py وليس من ملف صفحة منفرد.

Complete Code

```
# Folder structure:
# example_10/
# |-- app.py            <- run this
# `-- pages/
#     |-- 1_Settings.py
#     |-- 2_Data.py
#     `-- 3_Report.py

# app.py (Home page)
import streamlit as st

# Initialize shared variables
if "subject" not in st.session_state:
    st.session_state.subject = "Mathematics"
if "num_students" not in st.session_state:
    st.session_state.num_students = 30

st.title("Student Portal")
st.write("Use the sidebar to navigate.")

# pages/1_Settings.py
import streamlit as st

with st.form("settings"):
    subject = st.selectbox("Subject",
        ["Mathematics","Science","English"])
    students = st.slider("Students", 5, 100, 30)
    saved = st.form_submit_button("Save")

if saved:
    # Write to session state -> visible on other pages
    st.session_state.subject = subject
    st.session_state.num_students = students
    st.success("Saved!")
```

Key Functions

• pages/ folder — Put .py files here to create pages automatically

• 1_Page.py naming — Number prefix controls sidebar order

• st.session_state — How pages share data with each other

• st.set_page_config(...) — Set page title and icon at the top of each file

Student Exercise

• Create a 3-page app: Settings, Data Generator, Report

• Settings page: pick subject, number of students, passing score

• Data page: generate random scores based on settings, show as table

• Report page: show pass/fail counts, bar chart, written summary

14. Example 11 – Image Display

عرض الصور ومعالجتها

أهداف التعلّم:

• Display images from a URL, file upload, and NumPy array

• Show image information: size, channels, pixel range

• Apply image operations: grayscale, flip, rotate, blur, edge detection

• Display images in a grid with column layout

• Download processed images

شرح الفكرة وطريقة العمل:

الأهداف: عرض الصور من URL أو ملف مرفوع أو array NumPy، وإظهار معلومات الصورة مثل الحجم والقنوات ونطاق قيم Pixels، وتطبيق عمليات grayscale وflip وrotate وblur وdetection edge، وعرض الصور في Grid، وتنزيل الصور المعالجة. هذا المثال يمهّد للمثال 12؛ لأن رفع الصورة وتحويلها إلى NumPy وعرضها هي خطوات أساسية قبل التصنيف. st.image() تقبل path file أو URL أو Image PIL أو array. NumPy تكون صور Keras arrays NumPy بشكل (height, width, channels)، والقيم قد تكون 0–255 أو 0.0–1.0. مكتبة PIL توفر عمليات معالجة الصور.

Complete Code

```
import streamlit as st
import numpy as np
from PIL import Image, ImageFilter, ImageOps

# Display from URL
img_url = "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"
st.image(img_url, caption="Cat from Wikipedia", width=300)

# Upload and display
uploaded = st.file_uploader("Upload image",
   type=["jpg","png"])

if uploaded is not None:
   image = Image.open(uploaded).convert("RGB")
   img_array = np.array(image)
   st.image(image, width=400)
   st.write(f"Shape: {img_array.shape}")
   st.write(f"Range: {img_array.min()} to {img_array.max()}")

   # Apply operations
   op = st.selectbox("Operation",
      ["Grayscale","Flip H","Blur","Edges"])
   if op == "Grayscale":
      result = ImageOps.grayscale(image)
   elif op == "Flip H":
      result = ImageOps.mirror(image)
   elif op == "Blur":
      result = image.filter(ImageFilter.GaussianBlur(4))
   elif op == "Edges":
      result = image.filter(ImageFilter.FIND_EDGES)

   col1, col2 = st.columns(2)
   col1.image(image, caption="Original")
   col2.image(result, caption=op)
```

Key Functions

- st.image(image) — Display image; accepts URL, PIL, NumPy
- st.image(image, width=300) — Control display width in pixels
- Image.open(file).convert("RGB") — Open and convert to RGB
- ImageOps.grayscale(img) — Convert to grayscale
- ImageOps.mirror(img) — Flip horizontally
- img.filter(ImageFilter.GaussianBlur(4)) — Apply blur
- img.filter(ImageFilter.FIND_EDGES) — Detect edges
- np.array(image) / 255.0 — Normalize pixels to 0.0–1.0

Student Exercise

- Build an Image Comparison App: upload two images and show side by side
- Apply the same operation to both simultaneously with one selectbox
- Show a metric comparing their sizes (wider, taller, more pixels)

15. Example 12 – Full Image Classification App

تطبيق تصنيف الصور الكامل

الأهداف: رفع Dataset للصور في ملف ZIP، وتدريب 6 نماذج أو أكثر تشمل SVM وKNN وForest Random وANN وCNN ونماذج Pretrained، وتصنيف صورة جديدة باستخدام أي نموذج تم تدريبه، وعرض نتائج تقييم شاملة مثل Matrix Confusion وROC-AUC وPrecision-Recall وMetrics Heatmap.

App Structure

```
example_12/
|-- app.py                <- Home + ZIP dataset upload
|-- requirements.txt
`-- pages/
   |-- 1_Upload_Predict.py  <- Upload image -> prediction
   |-- 2_Train_Models.py    <- Train all models
   `-- 3_Results.py         <- All evaluation figures
```

تتكون الواجهة من الصفحة الرئيسية لرفع Dataset، وصفحة لرفع صورة والحصول على Prediction، وصفحة لتدريب النماذج، وصفحة لعرض كل نتائج التقييم.

Available Pretrained Models

يذكر الملف أن هناك 40 نموذجًا متاحًا إجمالًا، ويعرض بعض الأمثلة التالية. بقية القائمة (28 نموذجًا إضافيًا) موجودة في صفحة Models Train داخل التطبيق.

Model	Input Size	Weights	ImageNet Top-1
MobileNetV2	224px	14 MB	71.3%
EfficientNetB0	224px	29 MB	77.1%
EfficientNetB3	300px	48 MB	81.6%
EfficientNetV2S	384px	88 MB	83.9%
ResNet50	224px	98 MB	74.9%
ResNet50V2	224px	98 MB	76.0%
VGG16	224px	528 MB	71.3%
InceptionV3	299px	92 MB	77.9%
Xception	299px	88 MB	79.0%
DenseNet121	224px	33 MB	75.0%
NASNetMobile	224px	23 MB	74.4%
ConvNeXtTiny	224px	109 MB	81.3%

Results Page – All Figures

- Accuracy Cards — one metric card per model with delta vs best model
- Accuracy Bar Chart — all models ranked by test accuracy
- Confusion Matrix — interactive: raw counts or normalized percentages
- All Confusion Matrices — grid showing all models at once
- ROC-AUC Curves — per-class + macro average, all models compared
- Precision-Recall Curves — per-class Average Precision, mean AP comparison
- Per-Class Metrics — grouped bar: Precision / Recall / F1 per class
- Metrics Heatmap — all models x all metrics in one color-coded table
- Error Analysis — misclassification rate, confused pairs, distribution
- Final Summary Table — Accuracy, F1, ROC-AUC, mAP, Kappa, MCC

Dataset Preparation

يجب ضغط Dataset في ملف ZIP يحتوي على مجلد فرعي واحد لكل Class. يجب أن تكون مجلدات الفئات في المستوى الأعلى مباشرة داخل ZIP، وليس داخل مجلد إضافي.

```
my_dataset.zip
|-- cat/
|   |-- cat001.jpg
|   `-- cat002.jpg
`-- dog/
    |-- dog001.jpg
    `-- dog002.jpg
```

ملاحظة المصدر: Class folders must be at the TOP LEVEL of ZIP — not inside an extra subfolder.

مهم: الملف الأصلي يوضح هيكل التطبيق ومخرجات التقييم وأسماء النماذج، لكنه لا يحتوي على Code Complete لتنفيذ تدريب وتصنيف الصور بالكامل؛ لذلك لم أضف كودًا غير موجود في المصدر.

16. Deploying on GitHub & Streamlit Cloud

النشر على GitHub وStreamlit Cloud

تتيح Streamlit Cloud نشر تطبيق Streamlit مجانًا من مستودع GitHub خلال دقائق. يحصل التطبيق على Public URL يمكن فتحه من المتصفح.

Step 1 — Create a GitHub Account

اذهب إلى https://github.com وأنشئ حسابًا مجانيًا، ثم تحقق من البريد الإلكتروني.

Step 2 — Install Git

نزّل Git من https://git-scm.com على Windows، واستخدم git install brew على Mac، وعلى Ubuntu/Linux استخدم sudo apt install git. تحقق باستخدام git --version.

Step 3 — Create a New GitHub Repository

اذهب إلى https://github.com/new، وأنشئ Repository مثل streamlit-tutorial، واجعله Public للنشر المجاني المذكور في المحاضرة.

Step 4 — Add requirements.txt

أنشئ requirements.txt بجوار app.py ليعرف Cloud Streamlit الحزم المطلوبة.

```
# Windows: download from https://git-scm.com
# Mac:
brew install git
# Ubuntu/Linux:
sudo apt install git

# Verify:
git --version
```

```
# requirements.txt
streamlit
pandas
numpy
matplotlib
seaborn
Pillow
scikit-learn
tensorflow
opencv-python-headless
```

Step 5 — Push Your Code to GitHub

```
# Navigate to your project folder
cd my-streamlit-app

# Initialize git
git init

# Add all files
git add .

# Commit
git commit -m "Initial commit: add Streamlit tutorial"

# Connect to your GitHub repository
git remote add origin https://github.com/YOUR_USERNAME/streamlit-tutorial.git

# Push to GitHub
git push -u origin main
```

Step 6 — Deploy on Streamlit Cloud

- Go to https://share.streamlit.io
- Click "Sign in with GitHub" and authorize Streamlit
- Click "New app"
- Select your repository from the dropdown
- Set the Main file path — e.g. app.py or example_12/app.py
- Click "Deploy" and wait 1–2 minutes

بعد النشر يصبح التطبيق متاحًا على رابط يشبه:

```
https://your-username-streamlit-tutorial-app-xxxx.streamlit.app
```

16.2 Project Structure on GitHub

```
my-streamlit-app/           <- GitHub repository root
|-- app.py                  <- Main entry point
|-- requirements.txt        <- Package list for Streamlit Cloud
|-- README.md               <- Project description
`-- pages/
   |-- 1_Settings.py
   |-- 2_Data.py
   `-- 3_Results.py

# NEVER upload your dataset folder to GitHub!
# Use .gitignore to exclude it:
```

16.3 Step 7 — Create .gitignore

أضف الملف التالي لمنع رفع الملفات الكبيرة أو الحساسة بالخطأ:

```
# .gitignore
dataset/
*.zip
*.h5
*.keras
__pycache__/
*.pyc
.env
```

16.4 Updating Your App

كل مرة ترفع فيها تغييرات إلى GitHub، يقوم Cloud Streamlit بإعادة نشر التطبيق تلقائيًا.

```
# Make changes to your code, then:
git add .
git commit -m "Update: added new feature"
git push

# Streamlit Cloud detects the push and redeploys automatically
# Takes about 1 minute
```

16.5 Fixing the ModuleNotFoundError

إذا ظهر ModuleNotFoundError على Cloud Streamlit، فهذا يعني أن هناك Package ناقصة في requirements.txt. أضف الحزمة ثم ارفع التعديل إلى GitHub.

```
# Error example:
# ModuleNotFoundError: No module named "matplotlib"

# Fix: add the missing package to requirements.txt
# Then push to GitHub:
git add requirements.txt
git commit -m "Fix: add matplotlib to requirements"
git push

# Streamlit Cloud will reinstall and redeploy automatically
```

16.6 Common Deployment Issues & Fixes

Problem	Fix
ModuleNotFoundError	Add the missing package to requirements.txt and push
Pages not showing in sidebar	Ensure pages/ folder is next to app.py in the repo
App crashes on large dataset	Use st.file_uploader instead of local folder path
Slow app on Cloud	Reduce image size or add @st.cache_data decorator
Secret API keys in code	Use Streamlit Secrets Manager instead of hardcoding
App works locally but not Cloud	Check requirements.txt has every import you use

16.7 Local vs Cloud Differences

Feature	Local	Streamlit Cloud
Install packages	pip install X	Add X to requirements.txt
Dataset location	Local folder	Upload via st.file_uploader
File storage	Persists	Resets on each session (use cloud storage)
GPU	Your GPU if available	CPU only on free tier
Memory	Your RAM	1 GB RAM on free tier

17. Quick Reference Cheatsheet

المرجع السريع

هذا القسم يجمع الدوال والأنماط الأساسية كما وردت في المحاضرة، للرجوع إليها بسرعة أثناء كتابة التطبيق.

Display Functions

Function	What it shows / returns
st.title("text")	Biggest heading
st.header("text")	Section heading
st.subheader("text")	Smaller section heading
st.write("text or data")	Auto-detect: text, number, DataFrame, chart
st.markdown("bold**")**	Formatted Markdown text
st.code("x=1", language="python")	Syntax-highlighted code block
st.info / success / warning / error	Colored alert boxes
st.image(img, width=300)	Display image (URL, PIL, NumPy)
st.dataframe(df)	Interactive sortable table
st.table(df)	Static clean table
st.metric("label", value)	Dashboard number card
st.json(dict)	Pretty-printed dictionary
st.pyplot(fig)	Display Matplotlib/Seaborn figure

Widget Functions

Function	What it shows / returns
st.button("label")	True when clicked
st.slider("label", min, max, default)	Selected number
st.text_input("label")	Typed string
st.text_area("label")	Multi-line typed string
st.number_input("label", min, max)	Selected number (precise)
st.checkbox("label")	True or False
st.radio("label", [options])	Selected option string
st.selectbox("label", [options])	Selected option string
st.multiselect("label", [options])	List of selected options
st.select_slider("label", [options])	Selected option from list
st.file_uploader("label", type=[...])	File object or None
st.color_picker("label")	Hex color string
st.date_input("label")	datetime.date object

Layout Functions

Function	What it shows / returns
col1, col2 = st.columns(2)	Create 2 equal side-by-side columns
st.columns([1, 2, 1])	Columns with relative widths
with st.sidebar:	Content goes to left sidebar panel
tab1, tab2 = st.tabs(["A","B"])	Create clickable tabs
with st.expander("label"):	Collapsible hidden section
st.container()	Invisible grouping box
st.empty()	Placeholder to fill later
st.divider()	Horizontal separator line

Session State

if "key" not in st.session_state: st.session_state.key = value — Initialize once

st.session_state.key — Read stored value

st.session_state.key = new_val — Update stored value

st.rerun() — Force immediate rerun

Multi-Page Apps

• Folder name: Must be exactly pages/ (lowercase)

• Location: pages/ must be next to app.py

• Run command: Always streamlit run app.py — never a page file
• Order control: Name files 1_Page.py, 2_Page.py to control order
• Data sharing: Use st.session_state to pass data between pages
• Page config: Add st.set_page_config() at the TOP of each page file

Deployment Checklist

☐ app.py exists at the root of your GitHub repository
☐ requirements.txt lists every package you import
☐ pages/ folder is next to app.py (not inside a subfolder)
☐ .gitignore excludes dataset/ and *.zip
☐ No local file paths hardcoded (use st.file_uploader instead)
☐ Repository is set to Public on GitHub